

Blender + Sionna RT + Micro-Doppler 完整链路说明

目标：解释从 Blender 静态场景到 Sionna RT 静态信道，再到

`micro_doppler_modulator.py` 动态微多普勒调制的完整流程。

核心结论：**Blender/Sionna** 负责静态传播环境，`micro_doppler_modulator.py` 负责无人机旋翼的高速动态相位调制。

0. 总体理解

这套链路里有三个“无人机”的概念，需要分清楚：

层级	作用	是否真的运动	主要负责
Blender 无人机	可视化几何模型，也可提供静态反射/遮挡	否，导出后是静态网格	场景外观、静态几何
Sionna RT 中的 <code>drone_rx</code>	无人机机身中心位置，作为接收机或目标位置参考	可以更新位置，但通常静态求一次 RT	路径、CIR、RadioMap
<code>micro_doppler_modulator.py</code> 数学无人机	机身 + 旋翼叶片散射点模型	是，解析计算叶片旋转	微多普勒相位调制

最重要的思想是：

```
Blender 静态场景
-> Sionna RT 静态多径 / CIR
    -> micro_doppler_modulator.py 添加旋翼动态相位
        -> 动态 CIR / 复基带信号
            -> STFT
                -> Micro-Doppler Spectrogram (MDS)
```

也就是说，**不需要让 Blender 里的叶片真的高速旋转**。原因是微多普勒需要很高的时间采样率，例如 50 kHz，如果每个采样点都重新导出 PLY 并重新跑射线追踪，计算量会非常大。项目采用的是更实际的混合方法：静态环境由 Sionna RT 计算，旋翼运动由解析散射点模型计算。

1. Blender 建静态场景

1.1 这一阶段做什么

在 Blender 中建立：

- 地面
- 建筑物
- 墙体 / 障碍物
- 基站或雷达的可视化位置
- 无人机外观模型

这里的无人机可以建出来，但它主要用于：

1. 让场景可视化更直观；
2. 提供无人机机身、机臂等静态几何；
3. 标记无人机机身中心大概在哪里。

1.2 这一阶段不做什么

Blender 静态场景本身不会自动产生微多普勒。原因是：

- 导出的 PLY/XML 是静态网格；
- Sionna RT 看到的是固定几何；
- 固定几何没有叶片高速旋转；
- 没有时间变化就没有微多普勒频移。

因此，Blender 中的无人机不是微多普勒的直接来源。微多普勒来自后面的 `micro_doppler_modulator.py`。

2. 导出到 Sionna RT

2.1 两种导出路线

项目中 `step8_blender_scene_interface.py` 支持两类方式：

路线	格式	说明
推荐路线	Blender / Mitsuba XML	Blender 通过 Mitsuba 插件直接导出 XML，再由 <code>load_scene()</code> 加载
PLY 中转	PLY -> XML	Blender 导出 PLY，再通过 XML 包装为 Sionna/Mitsuba 场景

对应流程：

```
Blender 场景
-> .xml
-> sionna.rt.load_scene()
```

或：

```
Blender 场景
-> .ply 网格
-> generate_scene_xml.py
-> .xml
-> sionna.rt.load_scene()
```

2.2 Sionna RT 看到的是什么

Sionna RT 加载后，看到的是一个静态电磁传播场景，包括：

- 几何形状；
- 材料；
- 反射/透射/散射边界；
- 发射机和接收机；
- 载频、天线、路径求解器参数。

这一步的目标是让 Sionna 能够计算传播路径，而不是让无人机旋翼动起来。

3. 在 Sionna 中设置 BS 和 `drone_rx`

3.1 BS 是什么

BS 可以理解为基站、雷达或发射机。代码中通常用：

```
Transmitter(name="bs_0", position=[x, y, z], power_dbm=...)
```

如果做单基地雷达近似，也可以把雷达位置理解为发射/接收同址。

3.2 drone_rx 是什么

drone_rx 是 Sionna 场景里的接收机或无人机机身中心参考点：

```
Receiver(name="drone_rx", position=drone_pos)
```

它的作用是告诉 Sionna：

我要计算从 BS 到无人机中心附近的传播路径

3.3 位置必须统一

这是整条链路最容易出错的地方。你需要保证：

```
Blender 中无人机中心位置
    = Sionna 中 drone_rx.position
    = micro_doppler_modulator.py 中 body_position
```

例如：

```
drone_pos = [0.0, 0.0, 3.0]
radar_pos = [20.0, 20.0, 8.0]
```

那么 Sionna 中：

```
rx = Receiver(name="drone_rx", position=drone_pos)
```

modulator 中：

```
config = UAVMicroDopplerConfig(
    body_position=drone_pos,
    radar_position=radar_pos,
)
```

如果这两个位置不一致，就会出现：

```
Sionna 认为无人机在 A 点  
micro_doppler_modulator.py 认为无人机在 B 点
```

这样生成的 MDS 物理意义就不一致。

4. PathSolver 跑一次静态 RT

4.1 这一阶段输出什么

Sionna RT 的 `PathSolver` 负责计算静态传播路径：

```
solver = PathSolver()  
paths = solver(  
    scene,  
    max_depth=5,  
    los=True,  
    specular_reflection=True,  
    diffuse_reflection=True,  
    diffraction=False,  
    refraction=False,  
    synthetic_array=False,  
)
```

然后通过：

```
a_static, tau_static = paths.cir(  
    sampling_frequency=122.88e6,  
    normalize_delays=False,  
    out_type="numpy",  
)
```

得到静态 CIR。

4.2 `a_static` 和 `tau_static` 是什么

输出	含义
<code>a_static</code>	每条传播路径的复数路径系数，包含路径衰减和相位
<code>tau_static</code>	每条传播路径的传播时延

可以理解成：

`a_static`：这条路径有多强、相位是多少
`tau_static`：这条路径晚到多久

这一步仍然是静态的，没有旋翼微多普勒。

4.3 为什么只跑一次 RT

因为环境多径变化慢，而旋翼转动非常快。项目采用的近似是：

环境路径结构不变
叶片运动只改变目标回波的相位

所以可以先用 Sionna RT 求一次静态传播环境，再用解析模型生成高速动态调制。

5. `micro_doppler_modulator.py` 创建“数学无人机”

5.1 它创建的不是 Blender 网格

`micro_doppler_modulator.py` 创建的是一个散射点模型：

1 个机身散射点
8 个叶片尖端散射点

默认是四旋翼：

4 个旋翼 x 每个旋翼 2 个叶片 = 8 个叶片散射点

5.2 关键配置

```
config = UAVMicroDopplerConfig(  
    carrier_freq=28e9,  
    blade_radius=0.15,  
    rotation_freq=100.0,  
    n_rotors=4,  
    n_blades_per_rotor=2,  
    rotor_arm_length=0.175,  
    body_position=drone_pos,  
    body_velocity=[0.0, 0.0, 0.0],  
    radar_position=radar_pos,  
    sampling_rate=50e3,  
    obs_duration=0.2,  
)  
modulator = MicroDopplerModulator(config)
```

这些参数决定：

参数	作用
<code>carrier_freq</code>	载频，决定波长，影响微多普勒强度
<code>blade_radius</code>	叶片半径，影响叶片尖端速度和调制指数
<code>rotation_freq</code>	旋翼转速，决定边带间隔和峰值频偏
<code>body_position</code>	无人机机身中心位置，必须和 <code>drone_rx</code> 对齐
<code>radar_position</code>	雷达/基站位置，决定观测角和距离变化
<code>sampling_rate</code>	微多普勒信号采样率，必须足够高
<code>obs_duration</code>	观测时长，决定能看到多少个旋翼周期

5.3 它如何产生微多普勒

代码会计算每个时间点叶片尖端的位置：

```
blade_tip(t) = rotor_center + R [cos( $\omega t + \varphi$ ), sin( $\omega t + \varphi$ ), 0]
```

再计算叶片散射点到雷达的距离变化：

$d_k(t)$

距离变化导致相位变化：

$$\varphi_k(t) = -2\pi d_k(t) / \lambda$$

相位随时间变化，就等价于产生多普勒频移。

6. `modulate_cir()` 把静态 CIR 变成动态 CIR

6.1 输入输出

输入：

```
a_dynamic, tau_dynamic = modulator.modulate_cir(
    a_static,
    tau_static,
    t_vec,
)
```

其中：

变量	含义
<code>a_static</code>	Sionna RT 静态路径系数
<code>tau_static</code>	Sionna RT 静态路径时延
<code>t_vec</code>	微多普勒时间采样点
<code>a_dynamic</code>	加入微多普勒后的动态路径系数
<code>tau_dynamic</code>	动态 CIR 时延；当前近似中基本保持不变

6.2 核心公式

对于每个散射点，计算它相对机身中心的额外相位：

$$\Delta\varphi_k(t) = -2\pi [d_k(t) - d_{\text{body}}(t)] / \lambda$$

然后把所有散射点的贡献加起来：

$$\text{modulation}(t) = \sum A_k \exp(j \Delta\phi_k(t))$$

最后作用到静态 CIR：

$$a_{\text{dynamic}}(t) = a_{\text{static}} \times \text{modulation}(t)$$

6.3 这一步的物理意义

`a_static` 表示：

环境和目标中心产生的静态传播路径

`modulation(t)` 表示：

无人机旋翼叶片相对机身中心的高速相位变化

两者相乘表示：

静态多径环境 + 动态旋翼微多普勒

这就是项目的关键桥梁。

7. Sum paths 得到复基带信号

7.1 为什么要对路径求和

动态 CIR 里有多条路径，每条路径都是一个复数贡献。接收机最终收到的是所有路径的叠加：

$$s(t) = \sum_p a_p(t)$$

代码上通常是：

```
signal = np.sum(a_dynamic[:, 0, 0, 0, :, 0], axis=-1)
```

这里得到的 `signal` 是一个复基带时间序列：

```
signal[t] = I(t) + jQ(t)
```

7.2 这个信号包含什么

它同时包含：

- 静态环境路径强度；
- 多径相干叠加；
- 机身散射；
- 叶片散射；
- 微多普勒相位调制。

如果需要，也可以加入噪声，用于模拟不同 SNR 条件下的 MDS 清晰度。

8. STFT 得到 MDS

8.1 为什么用 STFT

微多普勒是时变频率现象，只看普通 FFT 只能看到整体频谱，不能看到频率随时间怎么变化。因此需要 STFT：

```
f, t, S_dB = modulator.stft_spectrogram(signal)
```

STFT 输出：

输出	含义
f	多普勒频率轴
t	时间轴
S_dB	时频谱功率

8.2 MDS 怎么看

MDS 中常见结构：

- 机身平动：接近水平线；
- 旋翼微多普勒：围绕中心频率展开的周期性结构；
- 旋翼转速越高：边带间隔越大；
- 叶片半径越大：频谱展开越宽；

- 载频越高：微多普勒越明显；
- SNR 越低：叶片弱回波越容易被淹没。

8.3 STFT 参数注意事项

参数	影响
<code>n_win</code>	窗长越大，频率分辨率越好，但时间分辨率越差
<code>overlap_ratio</code>	重叠越高，谱图越平滑，但计算量更大
<code>sampling_rate</code>	必须满足 Nyquist，否则微多普勒频率会混叠
<code>obs_duration</code>	观测越久，能看到更多旋翼周期

9. 推荐代码骨架

下面是从静态 Sionna RT 到 MDS 的最小流程骨架：

```
import numpy as np
from sionna.rt import load_scene, PlanarArray, Transmitter, Receiver, PathSolver
from micro_doppler_modulator import UAVMicroDopplerConfig, MicroDopplerModulator

# 1. 加载 Blender / XML 场景
scene = load_scene("your_blender_scene.xml")
scene.frequency = 28e9

# 2. 设置天线阵列
scene.tx_array = PlanarArray(
    num_rows=1, num_cols=1,
    vertical_spacing=0.5, horizontal_spacing=0.5,
    pattern="iso", polarization="V",
)
scene.rx_array = PlanarArray(
    num_rows=1, num_cols=1,
    vertical_spacing=0.5, horizontal_spacing=0.5,
    pattern="iso", polarization="V",
)

# 3. 设置位置
radar_pos = [20.0, 20.0, 8.0]
```

```
drone_pos = [0.0, 0.0, 3.0]
```

```
scene.add(Transmitter(name="bs_0", position=radar_pos, power_dbm=0.0))
```

```
scene.add(Receiver(name="drone_rx", position=drone_pos))
```

4. 静态 RT

```
solver = PathSolver()
```

```
paths = solver(  
    scene,  
    max_depth=5,  
    los=True,  
    specular_reflection=True,  
    diffuse_reflection=True,  
    diffraction=False,  
    refraction=False,  
    synthetic_array=False,  
)
```

```
a_static, tau_static = paths.cir(  
    sampling_frequency=122.88e6,  
    normalize_delays=False,  
    out_type="numpy",  
)
```

5. 数学无人机微多普勒模型

```
cfg = UAVMicroDopplerConfig(  
    carrier_freq=28e9,  
    blade_radius=0.15,  
    rotation_freq=100.0,  
    body_position=drone_pos,  
    radar_position=radar_pos,  
    sampling_rate=50e3,  
    obs_duration=0.2,  
)  
modulator = MicroDopplerModulator(cfg)
```

6. 静态 CIR -> 动态 CIR

```
t_vec = np.arange(int(cfg.sampling_rate * cfg.obs_duration)) / cfg.sampling_rate  
a_dynamic, tau_dynamic = modulator.modulate_cir(a_static, tau_static, t_vec)
```

7. 多路径叠加 -> 复基带信号

```
signal = np.sum(a_dynamic[:, 0, 0, 0, :, 0], axis=-1)

# 8. STFT -> MDS
f, t, S_dB = modulator.stft_spectrogram(signal)
```

10. 常见误区

误区 1: `micro_doppler_modulator.py` 创建了 Blender 无人机

不对。它创建的是数学散射点模型，不是 Blender 网格，也不是 Sionna 场景对象。

误区 2: Blender 中有无人机，就自动有微多普勒

不对。静态无人机网格只能产生静态反射，不能产生旋翼高速运动引起的频移。

误区 3: 必须逐帧旋转 PLY 才能模拟微多普勒

理论上可以，但工程上不划算。更推荐静态 RT + 解析微多普勒调制。

误区 4: `body_position` 可以随便填

不对。`body_position` 必须和 Sionna 中的 `drone_rx.position` 对齐，否则 RT 几何和微多普勒几何不一致。

误区 5: `radar_position` 可以随便选

不对。`radar_position` 应该对应实际 BS / radar 位置。它决定观测角和散射点距离变化，从而影响微多普勒强度。

11. 一句话总结

这条链路的本质是：

Blender 提供静态几何，Sionna RT 提供静态多径，`micro_doppler_modulator.py` 提供无人机旋翼的动态散射点相位调制，最后通过路径叠加和 STFT 得到 MDS。